

Create toy problem

Author Name

December 10, 2025

1 Create the Environment

To create a toy problem, you can start by creating the environment. For a 2D environment, you can extend `Environment2D`, otherwise you need to implement `MLKEnvironment`. `Environment2D` provides basic functionality and notably defines the step function which calls agents to perform actions.

1.1 Create State

State represents the environment's state at a given time. State should gather all the information necessary to describe the environment at a given time. There already exist an implementation of `State2DGridInt` that you can use if your environment is a 2D grid with information stored as integers. Otherwise, you need to create your own implementation of State that will depends on your toy problem.

When implementing State, you must redefine the `getObservation` method, which defines how the agent perceives the environment's state. `getObservation` returns a `Map` that maps all agents to their observations. For instance, `State2DGridInt`'s `getObservation` method returns a Map that links each agents to an `ObservationPositionsValues`, giving all informations in the neighborhood of the agent. This neighborhood can be defined during the instantiation of `State2DGridInt`.

1.2 Create Observation

Observation represents what the agent perceives from the environment's state. It can be a full or partial representation of the state, depending on whether the environment is fully observable or partially observable. To create an observation you need to extend `DefaultObservation`. Extending `DefaultObservation` requires defining the `add` and `equals` methods. `Equals` must be

redefined, otherwise policies that work with Qtables will not work properly because they will not be able to identify previously seen observations. When you override equals, it is also recommended to override hashCode.

The *add* method is used to combine two observations. To see an example of how to implement this method, you can refer to the class *ObservationPositionsValues*.

To be used as neural network input, observations must be converted to arrays of double. To do so, you need to implement a *WrapperObservationVector*.

1.3 Create Action

Action represents the actions that an agent can perform in the environment. To create an action, you need to extend *DefaultAction*. Action can be many things, an integer, a string, or a more complex object. 2 examples of actions are already implemented in marlkit and can be used in different toy problems: *Action2DMove* and *ActionInt*. The first one represents a movement in a 2D grid, and the second one represents an action as an integer. One common way of using the latter is: you have 3 possible actions, you can use *ActionInt*, the int representing the index of the action (0, 1 or 2).

2 Create the Launcher

To start the simulation of your toy problem, you need to create a launcher. A launcher extends *MLKLauncher*. *MLKLauncher* extends the MADKIT *SimuLauncher*. Your launcher must implement *onLaunchSimulatedAgents*. This method should create and launch the agents. The Environment, Scheduler and Viewer are selected in the annotation of the launcher class.

3 Create the Scheduler

The core structure of the Scheduler is already implemented in marlkit with the abstract class *MLKScheduler*. You should extends this class to specify the criteria for:

- Ending an episode,
- Start displaying the simulation,
- End displaying the simulation,

- Ending the simulation

To create your criteria, you can use the existing ones (*ReachTimeCriterion*, *ModuloTimeCriterion*, etc.), combine them (see *Criteria*) or create your own by implementing the *Criterion* interface.

The scheduler is responsible for calling the methods in correct order. By default, on activation, it makes the agents notify the environment. At each simulation step, it makes called `Environment.step()`, `agent.updatePolicy()`. At the end, it calls `onEnd()` methods for both the agents and the environment.

4 Create the Viewer

Viewer is responsible for displaying the environment. To create a viewer, you need to extend *Viewer* or *Viewer2D* that are MADKIT classes. Doing so, you need to implement the *render* method. With `getGraphics()` you can draw shapes, text, etc.

5 Agent

AgentStandard is a basic implementation of an *MLKAgent*. It was designed to be usable in most toy problems. If your toy problem requires specific agent behavior, you can extend *AgentStandard* and override the necessary methods, or redefine your own agent by implementing *MLKAgent*.

6 Common Problems

6.1 NoClassDefFoundError

It is possible to get this error when trying to launch our launcher:

```
Error: unable to load main class
marlkit.hello.LauncherTest in module marlkit.xp
java.lang.NoClassDefFoundError
```

In this case, you need to force the JVM (Java Virtual Machine) to load all project libraries at startup. Here are 2 solutions depending on the IDE

used:

VSCode In the `.vscode/launch.json` file, you need to add the option:

```
"vmArgs": "--add-modules ALL-MODULE-PATH"
```

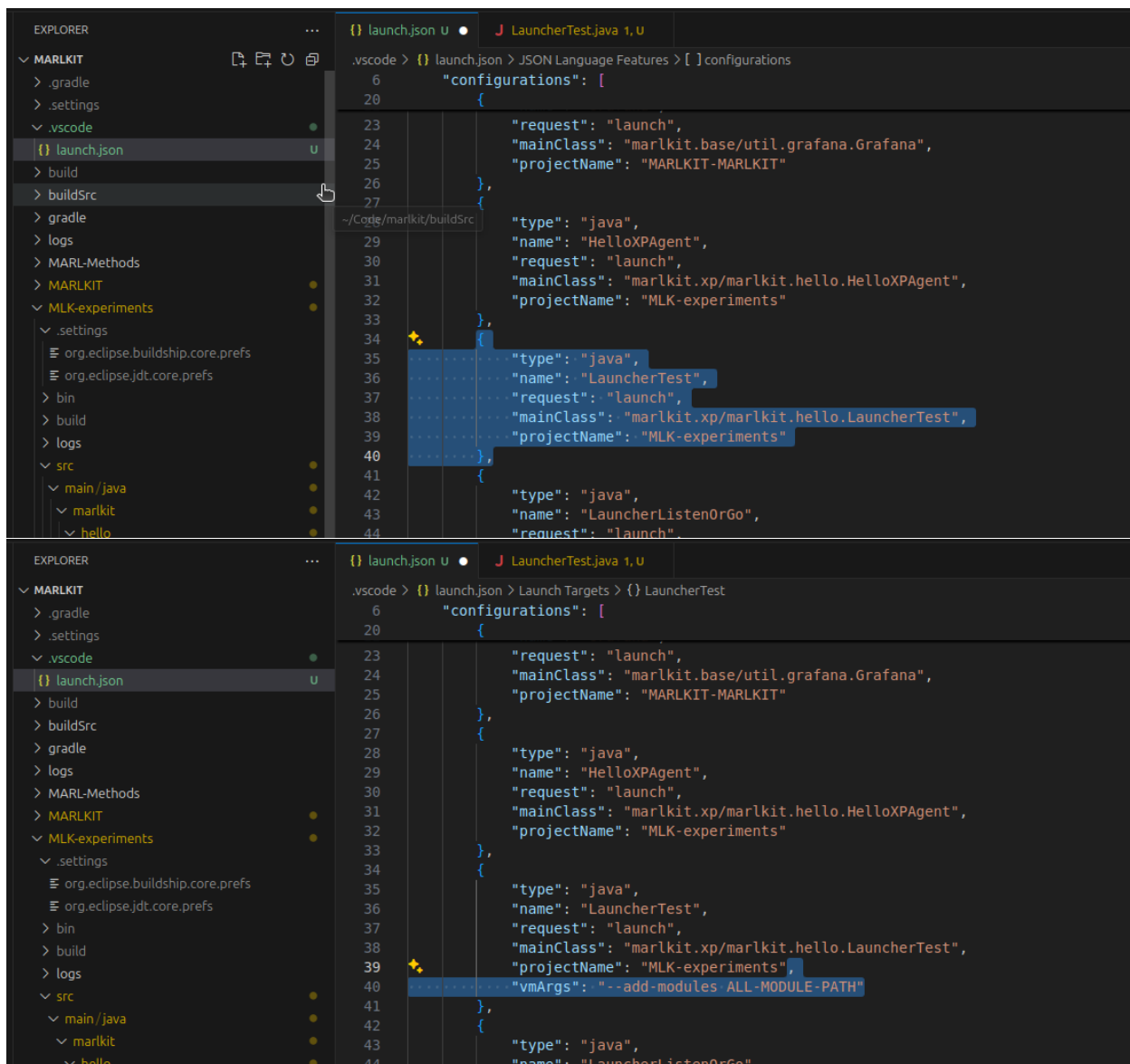


Figure 1: VSCode resolution for `NoClassDefFoundError`

Eclipse In Eclipse, go to run configurations, then in the arguments tab, and add to VM arguments:

```
--add-modules ALL-MODULE-PATH
```

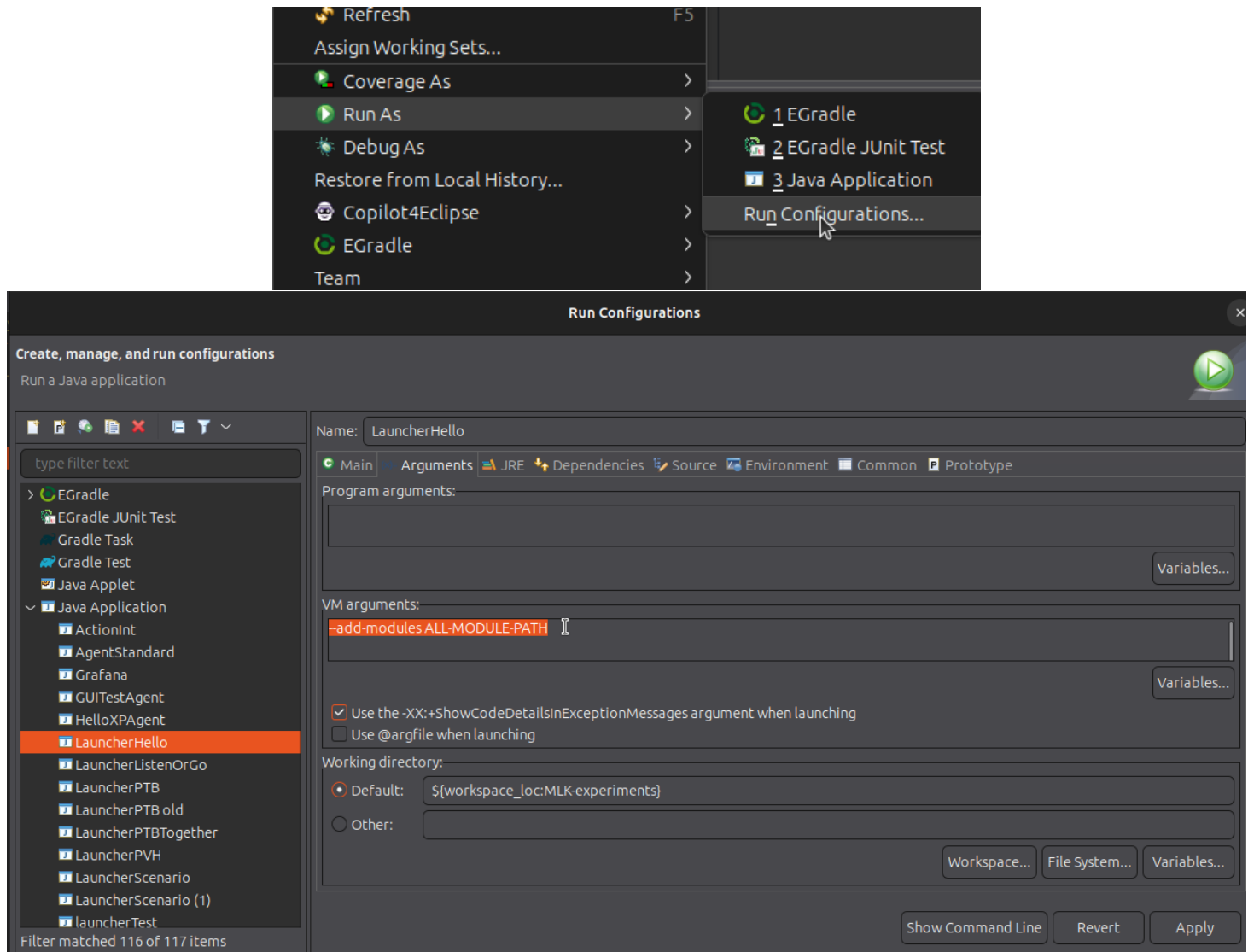


Figure 2: Eclipse resolution for NoClassDefFoundError